

8.4 Tree Traversal Algorithms

A **traversal** of a tree T is a systematic way of accessing, or “visiting,” all the positions of T . The specific action associated with the “visit” of a position p depends on the application of this traversal, and could involve anything from incrementing a counter to performing some complex computation for p . In this section, we describe several common traversal schemes for trees, implement them in the context of our various tree classes, and discuss several common applications of tree traversals.

8.4.1 Preorder and Postorder Traversals of General Trees

In a **preorder traversal** of a tree T , the root of T is visited first and then the subtrees rooted at its children are traversed recursively. If the tree is ordered, then the subtrees are traversed according to the order of the children. The pseudocode for the preorder traversal of the subtree rooted at a position p is shown in Code Fragment 8.12.

Algorithm preorder(p):

```

perform the “visit” action for position  $p$     { this happens before any recursion }
for each child  $c$  in children( $p$ ) do
    preorder( $c$ )                               { recursively traverse the subtree rooted at  $c$  }
```

Code Fragment 8.12: Algorithm preorder for performing the preorder traversal of a subtree rooted at position p of a tree.

Figure 8.13 portrays the order in which positions of a sample tree are visited during an application of the preorder traversal algorithm.

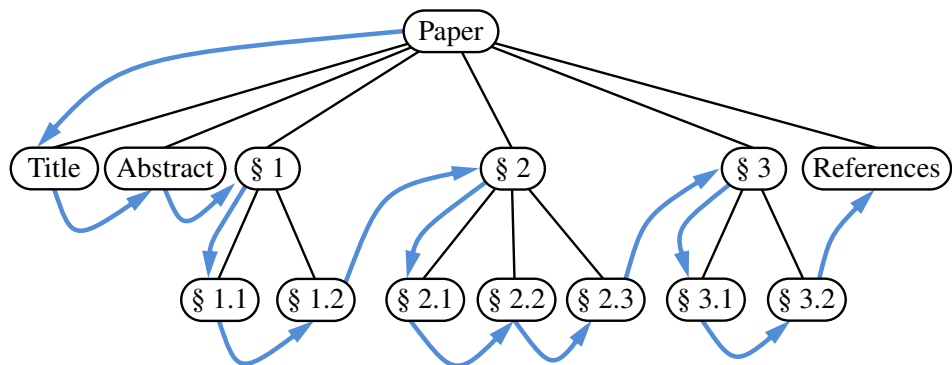


Figure 8.13: Preorder traversal of an ordered tree, where the children of each position are ordered from left to right.

Postorder Traversal

Another important tree traversal algorithm is the *postorder traversal*. In some sense, this algorithm can be viewed as the opposite of the preorder traversal, because it recursively traverses the subtrees rooted at the children of the root first, and then visits the root (hence, the name “postorder”). Pseudocode for the postorder traversal is given in Code Fragment 8.13, and an example of a postorder traversal is portrayed in Figure 8.14.

Algorithm postorder(p):

```

for each child  $c$  in children( $p$ ) do
    postorder( $c$ )           { recursively traverse the subtree rooted at  $c$  }
    perform the “visit” action for position  $p$  { this happens after any recursion }

```

Code Fragment 8.13: Algorithm postorder for performing the postorder traversal of a subtree rooted at position p of a tree.

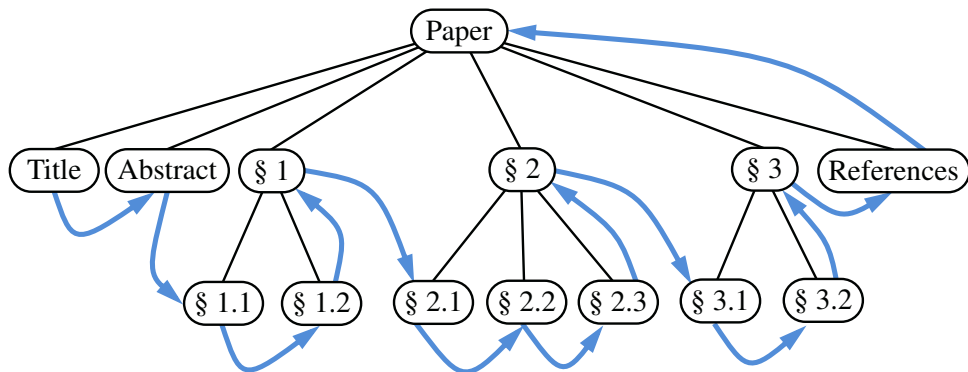


Figure 8.14: Postorder traversal of the ordered tree of Figure 8.13.

Running-Time Analysis

Both preorder and postorder traversal algorithms are efficient ways to access all the positions of a tree. The analysis of either of these traversal algorithms is similar to that of algorithm height, given in Code Fragment 8.5 of Section 8.1.3. At each position p , the nonrecursive part of the traversal algorithm requires time $O(c_p + 1)$, where c_p is the number of children of p , under the assumption that the “visit” itself takes $O(1)$ time. By Proposition 8.4, the overall running time for the traversal of tree T is $O(n)$, where n is the number of positions in the tree. This running time is asymptotically optimal since the traversal must visit all n positions of the tree.

8.4.2 Breadth-First Tree Traversal

Although the preorder and postorder traversals are common ways of visiting the positions of a tree, another approach is to traverse a tree so that we visit all the positions at depth d before we visit the positions at depth $d + 1$. Such an algorithm is known as a *breadth-first traversal*.

A breadth-first traversal is a common approach used in software for playing games. A *game tree* represents the possible choices of moves that might be made by a player (or computer) during a game, with the root of the tree being the initial configuration for the game. For example, Figure 8.15 displays a partial game tree for Tic-Tac-Toe.

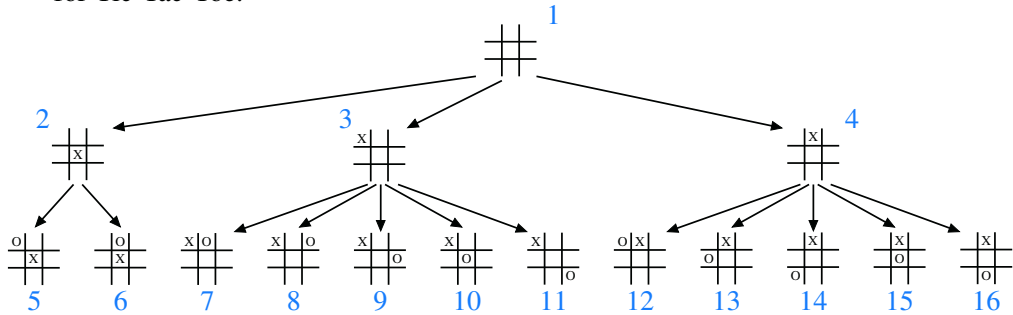


Figure 8.15: Partial game tree for Tic-Tac-Toe when ignoring symmetries; annotations denote the order in which positions are visited in a breadth-first tree traversal.

A breadth-first traversal of such a game tree is often performed because a computer may be unable to explore a complete game tree in a limited amount of time. So the computer will consider all moves, then responses to those moves, going as deep as computational time allows.

Pseudocode for a breadth-first traversal is given in Code Fragment 8.14. The process is not recursive, since we are not traversing entire subtrees at once. We use a queue to produce a FIFO (i.e., first-in first-out) semantics for the order in which we visit nodes. The overall running time is $O(n)$, due to the n calls to enqueue and n calls to dequeue.

Algorithm `breadthfirst()`:

Initialize queue Q to contain `root()`

while Q not empty **do**

$p = Q.dequeue()$ { p is the oldest entry in the queue }

 perform the “visit” action for position p

for each child c in `children( $p$ )` **do**

$Q.enqueue(c)$ { add p ’s children to the end of the queue for later visits }

Code Fragment 8.14: Algorithm for performing a breadth-first traversal of a tree.

8.4.3 Inorder Traversal of a Binary Tree

The standard preorder, postorder, and breadth-first traversals that were introduced for general trees can be directly applied to binary trees. In this section, we will introduce another common traversal algorithm specifically for a binary tree.

During an **inorder traversal**, we visit a position between the recursive traversals of its left and right subtrees. The inorder traversal of a binary tree T can be informally viewed as visiting the nodes of T “from left to right.” Indeed, for every position p , the inorder traversal visits p after all the positions in the left subtree of p and before all the positions in the right subtree of p . Pseudocode for the inorder traversal algorithm is given in Code Fragment 8.15, and an example of an inorder traversal is portrayed in Figure 8.16.

Algorithm `inorder( $p$ ):`

if p has a left child lc **then**

`inorder( $lc$ )`

 { recursively traverse the left subtree of p }

 perform the “visit” action for position p

if p has a right child rc **then**

`inorder( $rc$ )`

 { recursively traverse the right subtree of p }

Code Fragment 8.15: Algorithm `inorder` for performing an inorder traversal of a subtree rooted at position p of a binary tree.

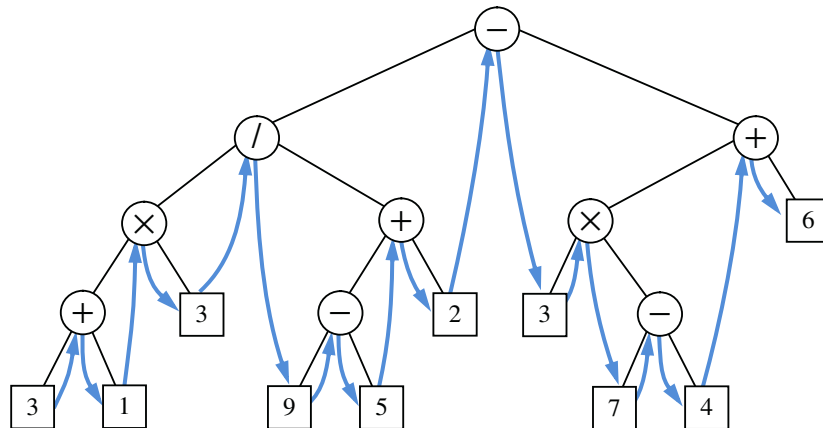


Figure 8.16: Inorder traversal of a binary tree.

The inorder traversal algorithm has several important applications. When using a binary tree to represent an arithmetic expression, as in Figure 8.16, the inorder traversal visits positions in a consistent order with the standard representation of the expression, as in $3 + 1 \times 3 / 9 - 5 + 2 \dots$ (albeit without parentheses).

Binary Search Trees

An important application of the inorder traversal algorithm arises when we store an ordered sequence of elements in a binary tree, defining a structure we call a **binary search tree**. Let S be a set whose unique elements have an order relation. For example, S could be a set of integers. A binary search tree for S is a proper binary tree T such that, for each internal position p of T :

- Position p stores an element of S , denoted as $e(p)$.
- Elements stored in the left subtree of p (if any) are less than $e(p)$.
- Elements stored in the right subtree of p (if any) are greater than $e(p)$.

An example of a binary search tree is shown in Figure 8.17. The above properties assure that an inorder traversal of a binary search tree T visits the elements in nondecreasing order.

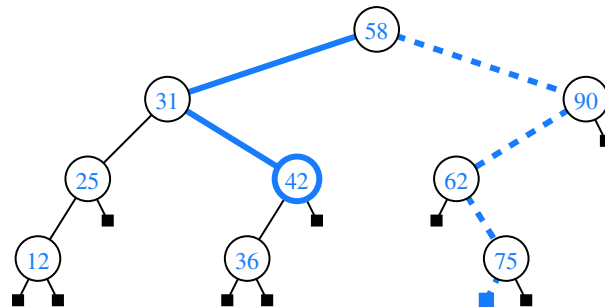


Figure 8.17: A binary search tree storing integers. The solid path is traversed when searching (successfully) for 42. The dashed path is traversed when searching (unsuccessfully) for 70.

We can use a binary search tree T for set S to find whether a given search value v is in S , by traversing a path down the tree T , starting at the root. At each internal position p encountered, we compare our search value v with the element $e(p)$ stored at p . If $v < e(p)$, then the search continues in the left subtree of p . If $v = e(p)$, then the search terminates successfully. If $v > e(p)$, then the search continues in the right subtree of p . Finally, if we reach a leaf, the search terminates unsuccessfully. In other words, a binary search tree can be viewed as a binary decision tree (recall Example 8.5), where the question asked at each internal node is whether the element at that node is less than, equal to, or larger than the element being searched for. We illustrate several examples of the search operation in Figure 8.17.

Note that the running time of searching in a binary search tree T is proportional to the height of T . Recall from Proposition 8.7 that the height of a binary tree with n nodes can be as small as $\log(n+1) - 1$ or as large as $n - 1$. Thus, binary search trees are most efficient when they have small height. Chapter 11 is devoted to the study of search trees.

8.4.4 Implementing Tree Traversals in Java

When first defining the tree ADT in Section 8.1.2, we stated that tree T must include the following supporting methods:

`iterator()`: Returns an iterator for all elements in the tree.

`positions()`: Returns an iterable collection of all positions of the tree.

At that time, we did not make any assumption about the order in which these iterations report their results. In this section, we will demonstrate how any of the tree traversal algorithms we have introduced can be used to produce these iterations as concrete implementations within the `AbstractTree` or `AbstractBinaryTree` base classes.

First, we note that an iteration of all *elements* of a tree can easily be produced if we have an iteration of all *positions* of that tree. Code Fragment 8.16 provides an implementation of the `iterator()` method by adapting an iteration produced by the `positions()` method. In fact, this is the identical approach we used in Code Fragment 7.14 of Section 7.4.2 for the `LinkedPositionalList` class.

```

1  //----- nested ElementIterator class -----
2  /* This class adapts the iteration produced by positions() to return elements. */
3  private class ElementIterator implements Iterator<E> {
4      Iterator<Position<E>> posIterator = positions().iterator();
5      public boolean hasNext() { return posIterator.hasNext(); }
6      public E next() { return posIterator.next().getElement(); } // return element!
7      public void remove() { posIterator.remove(); }
8  }
9
10 /** Returns an iterator of the elements stored in the tree. */
11 public Iterator<E> iterator() { return new ElementIterator(); }

```

Code Fragment 8.16: Iterating all elements of an `AbstractTree` instance, based upon an iteration of the positions of the tree.

To implement the `positions()` method, we have a choice of tree traversal algorithms. Given that there are advantages to each of those traversal orders, we provide public implementations of each strategy that can be called directly by a user of our class. We can then trivially adapt one of those as a default order for the `positions` method of the `AbstractTree` class. For example, on the following page we will define a public method, `preorder()`, that returns an iteration of the positions of a tree in preorder; Code Fragment 8.17 demonstrates how the `positions()` method can be trivially defined to rely on that order.

```
public Iterable<Position<E>> positions() { return preorder(); }
```

Code Fragment 8.17: Defining `preorder` as the default traversal algorithm for the public `positions` method of an abstract tree.

Preorder Traversals

We begin by considering the *preorder traversal* algorithm. Our goal is to provide a public method `preorder()`, as part of the `AbstractTree` class, which returns an iterable container of the positions of the tree in preorder. For ease of implementation, we choose to produce a *snapshot iterator*, as defined in Section 7.4.2, returning a list of all positions. (Exercise C-8.47 explores the goal of implementing a *lazy iterator* that reports positions in preorder.)

We begin by defining a private utility method, `preorderSubtree`, given in Code Fragment 8.18, which allows us to parameterize the recursive process with a specific position of the tree that serves as the root of a subtree to traverse. (We also pass a list as a parameter that serves as a buffer to which “visited” positions are added.)

```

1  /** Adds positions of the subtree rooted at Position p to the given snapshot. */
2  private void preorderSubtree(Position<E> p, List<Position<E>> snapshot) {
3      snapshot.add(p);      // for preorder, we add position p before exploring subtrees
4      for (Position<E> c : children(p))
5          preorderSubtree(c, snapshot);
6  }
```

Code Fragment 8.18: A recursive subroutine for performing a preorder traversal of the subtree rooted at position p of a tree. This code should be included within the body of the `AbstractTree` class.

The `preorderSubtree` method follows the high-level algorithm originally described as pseudocode in Code Fragment 8.12. It has an implicit base case, as the **for** loop body never executes if a position has no children.

The public `preorder` method, shown in Code Fragment 8.19, has the responsibility of creating an empty list for the snapshot buffer, and invoking the recursive method at the root of the tree (assuming the tree is nonempty). We rely on a `java.util.ArrayList` instance as an `Iterable` instance for the snapshot buffer.

```

1  /** Returns an iterable collection of positions of the tree, reported in preorder. */
2  public Iterable<Position<E>> preorder() {
3      List<Position<E>> snapshot = new ArrayList<>();
4      if (!isEmpty())
5          preorderSubtree(root(), snapshot);    // fill the snapshot recursively
6      return snapshot;
7  }
```

Code Fragment 8.19: A public method that performs a preorder traversal of an entire tree. This code should be included within the body of the `AbstractTree` class.

Postorder Traversal

We implement a *postorder traversal* using a similar design as we used for a pre-order traversal. The only difference is that a “visited” position is not added to a postorder snapshot until *after* all of its subtrees have been traversed. Both the recursive utility and the top-level public method are given in Code Fragment 8.20.

```

1  /** Adds positions of the subtree rooted at Position p to the given snapshot. */
2  private void postorderSubtree(Position<E> p, List<Position<E>> snapshot) {
3      for (Position<E> c : children(p))
4          postorderSubtree(c, snapshot);
5      snapshot.add(p);      // for postorder, we add position p after exploring subtrees
6  }
7  /** Returns an iterable collection of positions of the tree, reported in postorder. */
8  public Iterable<Position<E>> postorder() {
9      List<Position<E>> snapshot = new ArrayList<>();
10     if (!isEmpty())
11         postorderSubtree(root(), snapshot);    // fill the snapshot recursively
12     return snapshot;
13 }

```

Code Fragment 8.20: Support for performing a postorder traversal of a tree. This code should be included within the body of the AbstractTree class.

Breadth-First Traversal

On the following page, we will provide an implementation of the breadth-first traversal algorithm in the context of our AbstractTree class (Code Fragment 8.21). Recall that the breadth-first traversal algorithm is not recursive; it relies on a queue of positions to manage the traversal process. We will use the LinkedQueue class from Section 6.2.3, although any implementation of the queue ADT would suffice.

Inorder Traversal for Binary Trees

The preorder, postorder, and breadth-first traversal algorithms are applicable to all trees. The inorder traversal algorithm, because it explicitly relies on the notion of a left and right child of a node, only applies to binary trees. We therefore include its definition within the body of the AbstractBinaryTree class. We use a similar design to our preorder and postorder traversals, with a private recursive utility for traversing subtrees. (See Code Fragment 8.22.)

For many applications of binary trees (for example, see Chapter 11), an inorder traversal is the most natural order. Therefore, Code Fragment 8.22 makes it the default for the AbstractBinaryTree class by overriding the positions method that was inherited from the AbstractTree class. Because the iterator() method relies on positions(), it will also use inorder when reporting the elements of a binary tree.


```

1  /** Returns an iterable collection of positions of the tree in breadth-first order. */
2  public Iterable<Position<E>> breadthfirst() {
3      List<Position<E>> snapshot = new ArrayList<>();
4      if (!isEmpty()) {
5          Queue<Position<E>> fringe = new LinkedList<>();
6          fringe.enqueue(root());           // start with the root
7          while (!fringe.isEmpty()) {
8              Position<E> p = fringe.dequeue(); // remove from front of the queue
9              snapshot.add(p);                 // report this position
10             for (Position<E> c : children(p))
11                 fringe.enqueue(c);           // add children to back of queue
12         }
13     }
14     return snapshot;
15 }

```

Code Fragment 8.21: An implementation of a breadth-first traversal of a tree. This code should be included within the body of the `AbstractTree` class.

```

1  /** Adds positions of the subtree rooted at Position p to the given snapshot. */
2  private void inorderSubtree(Position<E> p, List<Position<E>> snapshot) {
3      if (left(p) != null)
4          inorderSubtree(left(p), snapshot);
5      snapshot.add(p);
6      if (right(p) != null)
7          inorderSubtree(right(p), snapshot);
8  }
9  /** Returns an iterable collection of positions of the tree, reported in inorder. */
10 public Iterable<Position<E>> inorder() {
11     List<Position<E>> snapshot = new ArrayList<>();
12     if (!isEmpty())
13         inorderSubtree(root(), snapshot); // fill the snapshot recursively
14     return snapshot;
15 }
16 /** Overrides positions to make inorder the default order for binary trees. */
17 public Iterable<Position<E>> positions() {
18     return inorder();
19 }

```

Code Fragment 8.22: Support for performing an inorder traversal of a binary tree, and for making that order the default traversal for binary trees. This code should be included within the body of the `AbstractBinaryTree` class.

8.4.5 Applications of Tree Traversals

In this section, we demonstrate several representative applications of tree traversals, including some customizations of the standard traversal algorithms.

Table of Contents

When using a tree to represent the hierarchical structure of a document, a preorder traversal of the tree can be used to produce a table of contents for the document. For example, the table of contents associated with the tree from Figure 8.13 is displayed in Figure 8.18. Part (a) of that figure gives a simple presentation with one element per line; part (b) shows a more attractive presentation, produced by indenting each element based on its depth within the tree.

Paper	Paper
Title	Title
Abstract	Abstract
§1	§1
§1.1	§1.1
§1.2	§1.2
§2	§2
§2.1	§2.1
...	...
(a)	(b)

Figure 8.18: Table of contents for a document represented by the tree in Figure 8.13: (a) without indentation; (b) with indentation based on depth within the tree.

The unindented version of the table of contents can be produced with the following code, given a tree *T* supporting the `preorder()` method:

```
for (Position<E> p : T.preorder())
    System.out.println(p.getElement());
```

To produce the presentation of Figure 8.18(b), we indent each element with a number of spaces equal to twice the element's depth in the tree (hence, the root element was unindented). If we assume that method, `spaces(n)`, produces a string of *n* spaces, we could replace the body of the above loop with the statement `System.out.println(spaces(2*T.depth(p)) + p.getElement())`. Unfortunately, although the work to produce the preorder traversal runs in $O(n)$ time, based on the analysis of Section 8.4.1, the calls to `depth` incur a hidden cost. Making a call to `depth` from every position of the tree results in $O(n^2)$ worst-case time, as noted when analyzing the algorithm `heightBad` in Section 8.1.3.

A preferred approach to producing an indented table of contents is to redesign a top-down recursion that includes the current depth as an additional parameter. Such an implementation is provided in Code Fragment 8.23. This implementation runs in worst-case $O(n)$ time (except, technically, the time it takes to print strings of increasing lengths).

```

1  /** Prints preorder representation of subtree of T rooted at p having depth d. */
2  public static <E> void printPreorderIndent(Tree<E> T, Position<E> p, int d) {
3      System.out.println(spaces(2*d) + p.getElement());    // indent based on d
4      for (Position<E> c : T.children(p))
5          printPreorderIndent(T, c, d+1);                  // child depth is d+1
6  }
```

Code Fragment 8.23: Efficient recursion for printing indented version of a preorder traversal. To print an entire tree T , the recursion should be started with form `printPreorderIndent(T, T.root(), 0)`.

In the example of Figure 8.18, we were fortunate in that the numbering was embedded within the elements of the tree. More generally, we might be interested in using a preorder traversal to display the structure of a tree, with indentation and also explicit numbering that was not present in the tree. For example, we might display the tree from Figure 8.2 beginning as:

```

Electronics R'Us
  1 R&D
  2 Sales
    2.1 Domestic
    2.2 International
      2.2.1 Canada
      2.2.2 S. America
```

This is more challenging, because the numbers used as labels are implicit in the structure of the tree. A label depends on the path from the root to the current position. To accomplish our goal, we add an additional parameter to the recursive signature. We send a list of integers representing the labels leading to a particular position. For example, when visiting the node *Domestic* above, we will send the list of values $\{2, 1\}$ that comprise its label.

At the implementation level, we wish to avoid the inefficiency of duplicating such lists when sending a new parameter from one level of the recursion to the next. A standard solution is to pass the same list instance throughout the recursion. At one level of the recursion, a new entry is temporarily added to the end of the list before making further recursive calls. In order to “leave no trace,” the extraneous entry must later be removed from the list by the same recursive call that added it. An implementation based on this approach is given in Code Fragment 8.24.

```

1  /** Prints labeled representation of subtree of T rooted at p having depth d. */
2  public static <E>
3  void printPreorderLabeled(Tree<E> T, Position<E> p, ArrayList<Integer> path) {
4      int d = path.size(); // depth equals the length of the path
5      System.out.print(spaces(2*d)); // print indentation, then label
6      for (int j=0; j < d; j++) System.out.print(path.get(j) + (j == d-1 ? " " : ". "));
7      System.out.println(p.getElement());
8      path.add(1); // add path entry for first child
9      for (Position<E> c : T.children(p)) {
10         printPreorderLabeled(T, c, path);
11         path.set(d, 1 + path.get(d)); // increment last entry of path
12     }
13     path.remove(d); // restore path to its incoming state
14 }

```

Code Fragment 8.24: Efficient recursion for printing an indented and *labeled* presentation of a preorder traversal.

Computing Disk Space

In Example 8.1, we considered the use of a tree as a model for a file-system structure, with internal positions representing directories and leaves representing files. In fact, when introducing the use of recursion back in Chapter 5, we specifically examined the topic of file systems (see Section 5.1.4). Although we did not explicitly model it as a tree at that time, we gave an implementation of an algorithm for computing the disk usage (Code Fragment 5.5).

The recursive computation of disk space is emblematic of a *postorder* traversal, as we cannot effectively compute the total space used by a directory until *after* we know the space that is used by its children directories. Unfortunately, the formal implementation of postorder, as given in Code Fragment 8.20, does not suffice for this purpose. We would like to have a mechanism for children to return information to the parent as part of the traversal process. A custom solution to the disk space problem, with each level of recursion providing a return value to the (parent) caller, is provided in Code Fragment 8.25.

```

1  /** Returns total disk space for subtree of T rooted at p. */
2  public static int diskSpace(Tree<Integer> T, Position<Integer> p) {
3      int subtotal = p.getElement(); // we assume element represents space usage
4      for (Position<Integer> c : T.children(p))
5          subtotal += diskSpace(T, c);
6      return subtotal;
7  }

```

Code Fragment 8.25: Recursive computation of disk space for a tree. We assume that each tree element reports the local space used at that position.

Parenthetic Representations of a Tree

It is not possible to reconstruct a general tree, given only the preorder sequence of elements, as in Figure 8.18a. Some additional context is necessary for the structure of the tree to be well defined. The use of indentation or numbered labels provides such context, with a very human-friendly presentation. However, there are more concise string representations of trees that are computer-friendly.

In this section, we explore one such representation. The *parenthetic string representation* $P(T)$ of tree T is recursively defined. If T consists of a single position p , then $P(T) = p.getElement()$. Otherwise, it is defined recursively as,

$$P(T) = p.getElement() + "(" + P(T_1) + ", " + \dots + ", " + P(T_k) + ")"$$

where p is the root of T and $T_1, T_2, \dots, T_k$ are the subtrees rooted at the children of p , which are given in order if T is an ordered tree. We are using “+” here to denote string concatenation. As an example, the parenthetic representation of the tree of Figure 8.2 would appear as follows (line breaks are cosmetic):

```
Electronics R'Us (R&D, Sales (Domestic, International (Canada,
S. America, Overseas (Africa, Europe, Asia, Australia))),
Purchasing, Manufacturing (TV, CD, Tuner))
```

Although the parenthetic representation is essentially a preorder traversal, we cannot easily produce the additional punctuation using the formal implementation of preorder. The opening parenthesis must be produced just before the loop over a position's children, the separating commas between children, and the closing parenthesis just after the loop completes. The Java method `parenthesize`, shown in Code Fragment 8.26, is a custom traversal that prints such a parenthetic string representation of a tree T .

```
1  /** Prints parenthesized representation of subtree of T rooted at p. */
2  public static <E> void parenthesize(Tree<E> T, Position<E> p) {
3      System.out.print(p.getElement());
4      if (T.isInternal(p)) {
5          boolean firstTime = true;
6          for (Position<E> c : T.children(p)) {
7              System.out.print( (firstTime ? " (" : ", ")); // determine proper punctuation
8              firstTime = false;                             // any future passes will get comma
9              parenthesize(T, c);                             // recur on child
10         }
11         System.out.print(")");
12     }
13 }
```

Code Fragment 8.26: Method that prints parenthetic string representation of a tree.

Using Inorder Traversal for Tree Drawing

An inorder traversal can be applied to the problem of computing a graphical layout of a binary tree, as shown in Figure 8.19. We assume the convention, common to computer graphics, that x -coordinates increase left to right and y -coordinates increase top to bottom, so that the origin is in the upper left corner of the drawing.

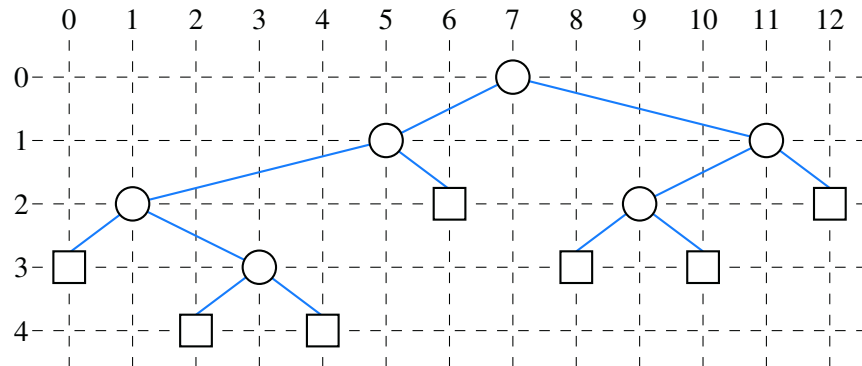


Figure 8.19: An inorder drawing of a binary tree.

The geometry is determined by an algorithm that assigns x - and y -coordinates to each position p of a binary tree T using the following two rules:

- $x(p)$ is the number of positions visited before p in an inorder traversal of T .
- $y(p)$ is the depth of p in T .

Code Fragment 8.27 provides an implementation of a recursive method that assigns x - and y -coordinates to positions of a tree in this manner. Depth information is passed from one level of the recursion to another, as done in our earlier example for indentation. To maintain an accurate value for the x -coordinate as the traversal proceeds, the method must be provided with the value of x that should be assigned to the leftmost node of the current subtree, and it must return to its parent a revised value of x that is appropriate for the first node drawn to the right of the subtree.

```

1  public static <E> int layout(BinaryTree<E> T, Position<E> p, int d, int x) {
2      if (T.left(p) != null)
3          x = layout(T, T.left(p), d+1, x);           // resulting x will be increased
4      p.getElement().setX(x++);                     // post-increment x
5      p.getElement().setY(d);
6      if (T.right(p) != null)
7          x = layout(T, T.right(p), d+1, x);         // resulting x will be increased
8      return x;
9  }
```

Code Fragment 8.27: Recursive method for computing coordinates at which to draw positions of a binary tree. We assume that the element type for the tree supports `setX` and `setY` methods. The initial call should be `layout(T, T.root(), 0, 0)`.

8.4.6 Euler Tours

The various applications described in Section 8.4.5 demonstrate the great power of recursive tree traversals, but they also show that not every application strictly fits the mold of a preorder, postorder, or inorder traversal. We can unify the tree-traversal algorithms into a single framework known as an **Euler tour traversal**. The Euler tour traversal of a tree T can be informally defined as a “walk” around T , where we start by going from the root toward its leftmost child, viewing the edges of T as being “walls” that we always keep to our left. (See Figure 8.20.)

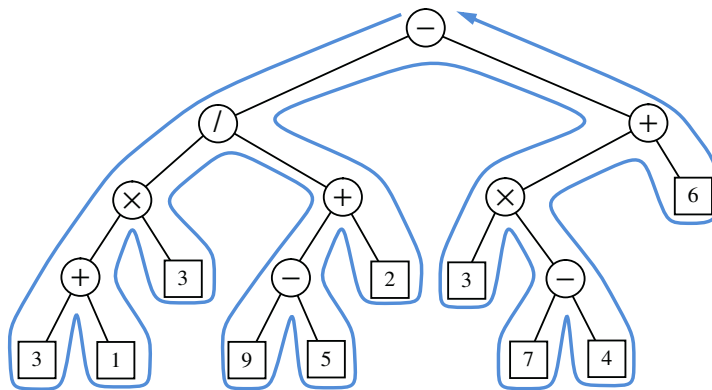


Figure 8.20: Euler tour traversal of a tree.

The complexity of the walk is $O(n)$, for a tree with n nodes, because it progresses exactly two times along each of the $n - 1$ edges of the tree—once going downward along the edge, and later going upward along the edge. To unify the concept of preorder and postorder traversals, we can view there being two notable “visits” to each position p :

- A “pre visit” occurs when first reaching the position, that is, when the walk passes immediately *left* of the node in our visualization.
- A “post visit” occurs when the walk later proceeds upward from that position, that is, when the walk passes to the *right* of the node in our visualization.

The process of an Euler tour can be naturally viewed as recursive. In between the “pre visit” and “post visit” of a given position will be a recursive tour of each of its subtrees. Looking at Figure 8.20 as an example, there is a contiguous portion of the entire tour that is itself an Euler tour of the subtree of the node with element “/”. That tour contains two contiguous subtours, one traversing that position’s left subtree and another traversing the right subtree.

In the special case of a binary tree, we can designate the time when the walk passes immediately *below* a node as an “in visit” event. This will be just after the tour of its left subtree (if any), but before the tour of its right subtree (if any).

The pseudocode for an Euler tour traversal of a subtree rooted at a position p is shown in Code Fragment 8.28.

Algorithm eulerTour(T, p):

```
perform the “pre visit” action for position  $p$ 
for each child  $c$  in  $T.children(p)$  do
    eulerTour( $T, c$ )           { recursively tour the subtree rooted at  $c$  }
perform the “post visit” action for position  $p$ 
```

Code Fragment 8.28: Algorithm eulerTour for performing an Euler tour traversal of a subtree rooted at position p of a tree.

The Euler tour traversal extends the preorder and postorder traversals, but it can also perform other kinds of traversals. For example, suppose we wish to compute the number of descendants of each position p in an n -node binary tree. We start an Euler tour by initializing a counter to 0, and then increment the counter during the “pre visit” for each position. To determine the number of descendants of a position p , we compute the difference between the values of the counter from when the pre-visit occurs and when the post-visit occurs, and add 1 (for p). This simple rule gives us the number of descendants of p , because each node in the subtree rooted at p is counted between p ’s visit on the left and p ’s visit on the right. Therefore, we have an $O(n)$ -time method for computing the number of descendants of each node.

For the case of a binary tree, we can customize the algorithm to include an explicit “in visit” action, as shown in Code Fragment 8.29.

Algorithm eulerTourBinary(T, p):

```
perform the “pre visit” action for position  $p$ 
if  $p$  has a left child  $lc$  then
    eulerTourBinary( $T, lc$ )           { recursively tour the left subtree of  $p$  }
perform the “in visit” action for position  $p$ 
if  $p$  has a right child  $rc$  then
    eulerTourBinary( $T, rc$ )           { recursively tour the right subtree of  $p$  }
perform the “post visit” action for position  $p$ 
```

Code Fragment 8.29: Algorithm eulerTourBinary for performing an Euler tour traversal of a subtree rooted at position p of a binary tree.

For example, a binary Euler tour can produce a traditional parenthesized arithmetic expression, such as " $((((3+1) \times 3) / ((9-5)+2)) - ((3 \times (7-4)) + 6))$ " for the tree in Figure 8.20, as follows:

- “Pre visit” action: if the position is internal, print “(”.
- “In visit” action: print the value or operator stored at the position.
- “Post visit” action: if the position is internal, print “)”.